AI-Driven RTO E-Challan System

Complete Project Execution Guide | Cloud (Render) & Localhost Environments

1. System Overview & Default Login Credentials

The system is an automated AI traffic enforcement platform. It ingests video feeds from RTSP cameras, mobile webcams, recorded clips, or a built-in synthetic generator. Violations (Speeding, No Helmet, Triple Riding, Red Light) are detected via Groq Vision AI, license plates read via ANPR, vehicle owners retrieved from MongoDB Atlas, SMS notifications delivered via Fast2SMS, and fines collected via Razorpay test gateway.

Role	Username	Password	Accessible Features
Administrator	**admin**	admin123	Full control: Camera management, AI settings, Rules, Users, Challans, Analytics
Traffic Officer	**officer1**	officer123	Operational control: Live monitoring, Manual review, Challan approval & status

2. Running the Cloud Deployment (Render.com)

Live Production URL: **https://ai-echallan-system-1.onrender.com**
The cloud instance serves the unified React frontend and FastAPI backend directly from Docker on Render.

Option A: Zero-Hardware Testing with Built-in Demo Camera (CAM-02)

- **Step 1:** Log in to **https://ai-echallan-system-1.onrender.com** with admin / admin123.
- **Step 2:** Navigate to **Live Monitoring** in the top navigation bar.
- **Step 3:** Under **CAM-02 (Marine Drive North)**, use the **'Demo Active' / 'Demo Paused'** toggle button to control traffic generation.
- **Step 4:** Simulated speeding cars and riders without helmets will trigger real-time AI detections and automatic challans.

Option B: Connecting a Real Mobile Phone Camera to the Cloud (Render)

Render is hosted in a cloud data center and cannot reach private local Wi-Fi IPs (like 10.173.33.193) directly. Use a free secure tunnel (Pinggy) from your laptop to expose the feed publicly:

1. Install **IP Webcam** from Google Play Store on your Android phone.
2. Connect your phone and laptop to the **same Wi-Fi or mobile hotspot**.
3. Open IP Webcam, tap **Start server**, and note the IP at the bottom (e.g. http://10.173.33.193:8080).
4. Open **PowerShell** on your laptop and run (zero install required):

```
$ ssh -p 443 -R0:10.173.33.193:8080 qr@a.pinggy.io
```

5. Pinggy displays an HTTPS URL, for example: https://random-subdomain.a.free.pinggy.link.
6. In the Render website, go to **Cameras** → Click **Edit** on **CAM-01**.
7. Set **Stream URL / RTSP URL** to: https://random-subdomain.a.free.pinggy.link/video.
8. Click **Test Connection** → It will report *'Connected and received a frame'* → Click **Save**.
9. Open **Live Monitoring**: Your physical mobile camera feed is now live on the Render cloud platform!

Option C: Direct Image & Video Upload

- Click **Upload Video / Image** in the navbar → Drag and drop recorded traffic footage for instant AI analysis.

3. Running the Project Locally (Laptop / PC)

For offline presentations or local testing with direct phone camera feeds (no tunneling needed):

Prerequisites

- **Python 3.10+** (with 'Add Python to PATH' enabled during installation).
- **Node.js 18+** and npm installed.
- **Git** installed.

One-Click Automated Launch (Recommended)

Double-click **START.bat** in the project root folder. It self-heals by validating or re-creating the virtual environment, installing requirements, building npm packages, and starting both servers automatically:

```
$ START.bat
```

• **Frontend:** http://localhost:5173 | **Backend API:** http://localhost:8000 | **Docs:** http://localhost:8000/docs

Direct Mobile Camera on Localhost (No Tunnel Required)

Because laptop and phone share the same Wi-Fi, you can use the raw local IP directly:

1. Open **http://localhost:5173** and log in as admin.
2. Go to **Cameras** → Click **Edit** on **CAM-01**.
3. Enter direct phone IP Webcam URL: http://10.173.33.193:8080/video → Click **Test Connection** → **Save**.

4. Complete End-to-End E-Challan Workflow

The following sequential steps demonstrate the entire lifecycle of an automated traffic violation:

Step	Stage	Process Description & Expected Output
1	Violation Ingestion	Live camera or CAM-02 detects a speed breach (>60 km/h), missing helmet, triple riding, or red light violation.
2	Vision AI Analysis	Groq Vision AI (Qwen 2.5-VL) validates the violation bbox and vehicle type from high-resolution frame crop.
3	ANPR Plate OCR	Plate Recognizer / EasyOCR extracts license number (e.g., **MH29CE7640**) with confidence score.
4	RTO Database Match	MongoDB Atlas matches registration to registered owner: **Nivedita Ingole** (Phone: **9370077413**).
5	SMS Dispatch	Fast2SMS sends automated SMS with challan number, fine amount, due date, and online payment URL.
6	Officer Verification	Traffic officer inspects evidence snapshot, radar speed trace, and timestamps on **E-Challans** page.
7	Citizen Online Payment	Click **Pay Now** → Razorpay test modal processes payment → Status changes to **PAID** with digital receipt.
8	Public Portal Lookup	Citizen opens /public → Enters vehicle **MH29CE7640** → Views and pays notice without logging in.

5. API Integrations & Cloud Infrastructure

All third-party services are active in backend/.env and MongoDB Atlas:

Service	Provider	Current State	Technical Purpose & Specifications
Vision AI	Groq Cloud	ACTIVE	qwen/qwen3.8-27b model for high-speed multi-violation detection
ANPR / OCR	Plate Recognizer	ACTIVE	Cloud API for Indian high-security registration plate character reading
Fallback OCR	OCR.Space	ACTIVE	Secondary OCR engine for adverse lighting or noisy plates
SMS Gateway	Fast2SMS	READY (Wallet)	v2 bulk API for automated SMS notifications (Requires Rs. 100 recharge)
Payment Gateway	Razorpay	ACTIVE (Test)	Standard Indian payment gateway for instant simulated fine collection
Database	MongoDB Atlas	ACTIVE (Cloud)	Cloud cluster holding vehicles, challans, cameras, users, and audit logs

FAST2SMS WALLET ACTIVATION CONFIRMATION

Fast2SMS Activation Note: Newly created Fast2SMS developer accounts require a one-time wallet recharge of **Rs. 100 INR** or more at fast2sms.com/dashboard to unlock programmatic API SMS dispatch. The project backend code is already fully configured and ready — the moment your Fast2SMS wallet has a balance of Rs. 100+, SMS alerts will dispatch automatically to the violator's phone with **zero code changes**.

6. Frequently Asked Questions & Troubleshooting

Q1: Why did Render show 'Could not connect to http://10.173.33.193:8080/video'?

Ans: 10.173.33.193 is a private IP inside your local Wi-Fi router. Render is hosted in a public cloud data center and has no route into private home Wi-Fi networks. Solution: Use the Pinggy command (ssh -p 443 -R0:... qr@a.pinggy.io) to forward the feed to a public HTTPS link, or run locally via START.bat.

Q2: Do I need to change the phone IP address every time?

Ans: If your phone stays connected to the same Wi-Fi / hotspot, the IP usually remains the same. If you restart your phone or change Wi-Fi networks, check the IP displayed at the bottom of the IP Webcam app screen (e.g. http://10.173.33.205:8080) and update the IP in your Pinggy command accordingly.

Q3: Does the Pinggy web URL change every time I run it?

Ans: Yes. Free Pinggy URLs are temporary and regenerate each time you re-open PowerShell and start the command. Whenever you start a new session, copy the new URL into **Cameras** → **CAM-01** → **Stream URL**.

Q4: What if the browser shows '500 Proxy Error' or 'Connection Refused' locally?

Ans: This means the backend server is not running on port 8000. Simply double-click START.bat. It will launch both backend and frontend together with automatic dependency resolution.

Q5: Can I demonstrate the project without any phone camera connected?

Ans: Yes! Simply use **CAM-02 (Demo Mode)** or the **Upload Video / Image** page. CAM-02 runs natively both in the cloud and locally, generating real-time traffic and violations with zero hardware requirements.